LAB REPORT

Fundamentals of Opera ng Systems (OS)

Assignment – 2

Name: DIPANSHU OHLYAN
Roll No: 2401201062
Course: BCA (AI & DS) [H]
Sec on: A

---

1. Introduc on

Deadlock is a cri cal problem in opera ng systems where a set of processes are unable to proceed because each process is wai ng for resources held by other processes. This situa on can halt system execu on and reduce efficiency.

To prevent deadlocks, opera ng systems use various techniques, one of which is the Banker's Algorithm. It is a deadlock avoidance algorithm that ensures the system always remains in a safe state, meaning all processes can complete execu on without leading to deadlock.

In this assignment, Banker's Algorithm has been implemented using Python to determine whether a system is in a safe or unsafe state and to iden fy the safe sequence of execu on.

---

2. Objec ves

- To understand the concept of deadlock and its impact
- To implement Banker's Algorithm using Python
- To calculate the Need Matrix
- To determine whether the system is in a safe state
- To find the safe sequence of process execu on
- To analyze system behavior based on resource alloca on

---

3. Tools and Technologies Used

- Programming Language: Python 3.x

- Code Editor: Visual Studio Code
- Platform: Windows
- Libraries Used: Built-in Python functions

**4. System Input and Data Representation**

**4.1 Overview**

The system is defined using:

- Number of processes
- Number of resource types
- Allocation Matrix
- Maximum Matrix
- Available Resources

These components represent the current state of resource allocation in the system.

**4.2 Allocation Matrix**

The Allocation Matrix shows the number of resources currently allocated to each process

```
    allocation = []
    print("\nEnter Allocation Matrix:")

    for i in range(n):
        row = list(map(int, input(f"P{i}: ").split()))
        allocation.append(row)


    print("\nAllocation Matrix:")
    print("-" * 30)
    for i in range(n):
        print(f"P{i}: {allocation[i]}")
```

✓ 21.8s

```
Enter Allocation Matrix:

Allocation Matrix:
------------------------------
P0: [0, 1, 0]
P1: [2, 0, 0]
P2: [3, 0, 2]
P3: [2, 1, 1]
P4: [0, 0, 2]
```

### 4.3 Maximum Matrix

The Maximum Matrix defines the maximum demand of each process for each resource type.

```
maximum = []
print("\nEnter Maximum Matrix:")

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    maximum.append(row)


print("\nMaximum Matrix:")
print("-" * 30)
for i in range(n):
    print(f"P{i}: {maximum[i]}")
```

✓ 16.0s

```
Enter Maximum Matrix:

Maximum Matrix:
------------------------------
P0: [7, 5, 3]
P1: [3, 2, 2]
P2: [9, 0, 2]
P3: [2, 2, 2]
P4: [4, 3, 3]
```

### 4.4 Available Resources

The Available vector represents the number of resources currently available in the system.

```
print("\nAvailable Resources:")
print("-" * 25)
for i, val in enumerate(available):
    print(f"R{i}: {val}")
```

✓ 0.0s

```
Available Resources:
-------------------------
R0: 3
R1: 3
R2: 2
```

## 5. Need Matrix Calculation

### 5.1 Formula

The Need Matrix represents the remaining resource requirement of each process and is calculated using:

$$\textbf{Need} = \textbf{Maximum} - \textbf{Allocation}$$

### 5.2 Implementation

Each element of the Need Matrix is calculated by subtracting the Allocation value from the Maximum value for the corresponding process and resource.

### 5.3 Output

The Need Matrix is displayed in a structured tabular format for clarity

```
    print("\nNeed Matrix:")
    print("-" * 30)

    for i in range(n):
        print(f"P{i}: {need[i]}")
```

✓ 0.0s

```
Need Matrix:
------------------------------
P0: [7, 4, 3]
P1: [1, 2, 2]
P2: [6, 0, 0]
P3: [0, 1, 1]
P4: [4, 3, 1]
```

## 6. Banker's Safety Algorithm

### 6.1 Working Principle

The Banker's Algorithm checks whether the system is in a safe state by simulating process execution based on available resources.

### 6.2 Steps Involved

1. **Initialize:**
    - Work = Available
    - Finish = False for all processes
2. **Find a process such that:**

- Need ≤ Work

3. **If found:**
    - Allocate resources to the process
    - Update Work
    - Mark process as finished
4. **Repeat until:**
    - All processes are completed OR
    - No further allocation is possible

### 6.3 Key Logic

The condition used to check whether a process can execute is:

- **Need ≤ Available resources**

This ensures that the process can complete without causing deadlock.

```
finish = [False] * n
safe_sequence = []
work = available.copy()

while len(safe_sequence) < n:
    found = False

    for i in range(n):
        if not finish[i]:

            if all(need[i][j] <= work[j] for j in range(m)):

                print(f"\nProcess P{i} is executing...")

                for j in range(m):
                    work[j] += allocation[i][j]

                safe_sequence.append(i)
                finish[i] = True
                found = True

                print(f"Updated Available (Work): {work}")

    if not found:
        break
```

✓ 0.0s